

Project Report

Learn and Contribute to Society





Final Project Report

Classifying Risk and Predicting Delay for TTC.

Arbert Owusu, Jeryshan Varatheswaran



Owusu, Arbert; Varatheswaran, Jeryshan first name of authors

Table of contents

Preface	1
Abstract	2
1 Introduction	3
1.1 Overview	3
1.1.1 Existing System	3
1.2 Objectives of Project (Must be clearly, precisely defined and must be covered in the work)	3
2 Pre-Processing and Exploratory Data Analysis	4
2.1 Dataset Collection	4
2.2 Data Pre-processing	4
2.3 Exploratory Data Analysis and Visualisations	5
2.4 Other Related Sections (Optional)	6
3 Methodology	7
3.1 Introduction to Python for Machine Learning	7
3.2 Platform and Machine Configurations Used	7
3.3 Data Split	7
3.4 Model Planning	7
3.5 Model Training:	8
3.6 Model Evaluation	8
3.7 Model Optimization	9
3.8 Final Model Building	9
4 Results	10
4.1 Description of the Models	10
4.2 Performance Metrics	11
4.3 Results Table	11

TABLE OF CONTENTS

ii

4.4	Interpretation of the Results	12
4.5	Visualization	13
4.6	Sensitivity Analysis	20
5	Conclusion	23
	References	24

List of Figures

4.1	Displays accuracy and precision scores for each classification model.	12
4.2	Displays MSE, RMSE, and R^2 for each regression model.	12
4.3	20-fold cross-validation results for the tuned decision tree model using grid search. .	13
4.4	Decision Tree Predictions	14
4.5	Linear Regression Actual vs Predicted Scatterplot	15
4.6	Linear Regression Residuals	16
4.7	Reg-Decision Tree Actual vs Predicted Scatterplot	17
4.8	Reg-Decision Tree Residuals	18
4.9	Random-reg Actual vs Predicted Scatterplot	19
4.10	Random-reg Tree Residuals	20
4.11	Reg-Sens	21
4.12	Displays the parameters used for <code>maximum_depth_range</code> and <code>minimum_sample_list</code> in each figure undergoing 20-fold cross validation grid search	21
4.13	First set of parameters, results of grid search using a specific set of parameters mentioned in Table	21
4.14	Second set of parameters, results of grid search using a specific set of parameters mentioned in Table	22

List of Tables

Preface

The problem we are trying to address are inconsistent times in bus timing, so we aim to predict with our ML model the delay risk and the bus delay in minutes. Even though there are systems that already exist that predict delays, with our model we aim to transform our data and use suitable algorithms to accurately predict delays for City of Toronto TTC bus delays.

The objective is to make our application useful in accurately predicting delay and delay risk in real-world scenarios. This can be achieved by introducing and training our data on newer updated data, so our model can be tweaked to fit with more relevant data. Ultimately, our main goal for this project is to predict delay risk within the City of Toronto for TTC bus delay data. To do so, we plan to create two models using regression and classification where one model displays how many minutes transit is delayed at a station while our classification model displays the percentage of delay risk in each month, stating which months have a delay risk of 50% or greater or less than 50%. Overall, these models can be utilized together and be further expanded upon to predict delays more precisely to the Toronto Transit Commission (TTC).

Abstract

In this study the investigation of multiple machine learning models is conducted on a transportation delay dataset. This dataset is characterized by large outliers and high variability within affecting the choice of models suitable for this data. For evaluation regression and classification models that fall under linear models and tree-based models and ensemble methods were used to predict the delay in minutes and the binary classification risk. Assessing these models, performance metrics RMSE, MSE, R^2 , accuracy, precision score alongside other metrics were used to compare performance. The results was that the tree-based models performed greater; specifically the decision tree classifier for predicting delay risk and random forest regressor for predicting delay in minutes. The random forest had a substantially lower error and higher explanation while decision tree for classification achieving the highest accuracy showcasing it's capability to better capture this data's structure. This is suggested by the complexity of the dataset being non-linear, further aligning with tree-based models handling complex data patterns through splitting to effectively reduce variance. Final thoughts, these results showcase the importance of selecting the right models given the context of the problem and the type of data interacted with, proving more effective techniques to solve the problem.

Chapter 1

Introduction

See this how you can add a reference, Knuth (1984).

(Pedregosa et al., 2011)

(Sehra, 2026)

1.1 Overview

1.1.1 Existing System

1.2 Objectives of Project (Must be clearly, precisely defined and must be covered in the work)

Chapter 2

Pre-Processing and Exploratory Data Analysis

Once you have your dataset, you need to preprocess it to make it suitable for machine learning. This includes cleaning the data, handling missing values, encoding categorical variables, and scaling numerical features. This step is crucial as the accuracy of your model will depend on the quality of your data.

2.1 Dataset Collection

The dataset we chose to use was the City of Toronto “TTC Bus Delay Data” 2025 dataset, as our objective for this project was aimed to relate to a real-world issue. The goal was to predict delay risk of a particular TTC vehicle on a specified day and also to predict the amount delay in minutes based off the datasets predictors and our own custom features. Finding the dataset was simple as city of Toronto contains many publically available data that is regularly updated,

Here is the city of Toronto Dataset (2025) Dataset Source

2.2 Data Pre-processing

Some issues to consider to be included: - Are there missing values in your data? Is there any pattern in whether or not a particular value is missing? How will the missing values be dealt with?

- Are there problems because of data size (rows or columns)? Are some algorithms more affected? Can you come up with any solutions?
- If you are doing any clustering, are there clusters in the data? How many? How well separated are they? Do different methods agree on the number of clusters?
- Is scaling of the variables an issue? For what methods?
- Are there outliers in the data? How did you deal with them?

- Does transforming the data simplify any analysis?

To pre-process the dataset we first went ahead and dropped columns with null/none values. Afterwards, we realized if we wanted to utilize classification methods such as logistic regression that we'd have to encode the data for each unique label/category within a column. Encoding columns such as bounds was straight forward as we could encode each direction of north, east, south, and west to a specific integer. However, this would be impossible to do for columns such as stations, transit codes, and lines within our dataset as there could be dozens of unique points of data. For example, there may be dozens or hundreds of stations present in the TTC. To solve this, we conducted research and found that `factorize`, which is a built-in method of the pandas library, can be utilized to automatically encode each unique categorical piece of data in these columns with a unique integer. By looking through the documentation of this built-in method, we were able to apply this and encode the labels for these datasets. To sum things up, we dropped null values and used the built-in method `factorize` to preprocess our dataset for model training and testing.

Furthermore, we did encounter issues using this preprocessed dataset. We found that there were several outliers that completely skewed our data during model training and testing. For example, there were lots of outliers within our "Delay Minutes" column of data within the dataset as lots of outliers were too long such as hitting 90 minutes or so when most of the data within this column was between 15 and 60 minutes. We dealt with these outliers by restricting the amount of the data for this column by only including values between 15 and 60 minutes. As a result, the data became less skewed and normalized. Overall, we ended up using this final preprocessed dataset to conduct model training, testing, and validation.

2.3 Exploratory Data Analysis and Visualisations

For exploratory data analysis and visualizations we utilized visualizations to examine the relationship between our models. Specifically for the regression models we plotted the predicted vs the actual data and a plot of the residuals to capture where our models struggled to find a relationship.

For the classifications models we utilized tree plots to plot what the decision tree looked like after splits and applying `kfold` - cross-validation. Scatterplots were also utilized to plot the actual delay-risk compared to our models predicted delay-risk.

- Data Visualizations: The graphs we created were to understand the patterns and trends in the data. The provided visualizations tools:
 - Scatterplots were used to examine the relationship between two variables, the actual and the predicted.
 - Basic plot functions was used to examine a single model predictions or to plot decision trees.
 - We also utilized basic plot functions to compare our models before tuning vs after tuning. This was used for classification trees to see the accuracy before and after applying k-fold cross validation.
- Statistical analysis: The linear regression model was used to analyze if the data had a linear relationship. To evaluate this we used MSE, RMSE and R^2 to evaluate meaningfully the error of our prediction of delay in minutes and how much of the data it actually captures. For the decision tree regressor we evaluated using the same metrics as linear regression to compare

the models. However with regression tree's we saw significant benefits from hyperparameter tuning by running grid-search to find optimal parameters we saw lower error and a higher r^2 relationship. Random forest regressor also saw benefits from hyperparameters-tuning resulting in a lower-error and better r^2 . From comparing these models we learned that distribution of the data was not linear but non-linear likely due to significant outliers and variances within the data.

- Feature selection: For feature selection we used: Line, Day, Station, Code, Bound, Vehicle, Time_Group_late, night, Time_Group_early morning, Time_Group_noon, Time_Group_afternoon, Time_Group_evening, Time_Group_night, Day_Of_Week, Hour, Month and Rush_Hour. For the features Time_Group, Day, Hour, Month, Day_of_Week and Rush_Hour we used existing features to extract these features. As we concluded these would be meaningful and predictive in classifying risk and predicting the delay in minutes. We found these variables to be predictive, so we removed existing features after extracting more predictive features from them after feature engineering.
- Dimensionality reduction: We did not use dimensionality reduction.

2.4 Other Related Sections (Optional)

Chapter 3

Methodology

3.1 Introduction to Python for Machine Learning

3.2 Platform and Machine Configurations Used

Google Colab and for machine configurations we used VS Code, Windows 11 os, Python 3.12.10, scikit-learn libraries, Panda and Numpy.

3.3 Data Split

Data splitting is the process of dividing the data into training, validation, and test sets. This step can help you evaluate the performance of the model on new data.

For data-splitting we did train/test split of 80/20, 80% of data being used for the training set and the 20% used for the testing set. We chose this split as a 80/20 split finds a balance between having enough data to train a strong model, but also enough data to test how it performs with new data.

3.4 Model Planning

The next step is to select the appropriate machine learning or deep learning models (2 - 3) that can be used to solve the problem. This involves evaluating different models and selecting the one that is best suited for the problem.

For predicting the delay_minutes a continuous metric chose was the models linear regression, decision tree regressor and random forest regressor. We chose the linear regression model to see how the model would perform on linear data, we assumed that TTC delay would not be linearly seperable due to sporadic events of transportation. However, wanted a base to see how that model would measure the data. Our second choice was decision tree regressor which splits data into subgroups based on the provided features. Lastly, we chose the random tree regressor model to see if it could capture variability in TTC data by leveraging the averaging of results to improve predictability.

Furthermore, to model this problem we planned to use several classification models and test to see which models yield the highest accuracy. Models we utilized involved linear discriminant analysis (LDA), quadratic discriminant analysis (QDA), logistic regression, decision tree, k-nearest neighbour (KNN).

3.5 Model Training:

This involves using the training data to adjust the model parameters to improve its accuracy.

For training the regression models we trained it using the parameters: Line, Day, Station, Code, Bound, Vehicle, Time_Group_late, Night, Time_Group_early, Morning, Time_Group_noon, Time_Group_afternoon, Time_Group_evening, Time_Group_night, Day_Of_Week, Hour, Month and Rush_Hour. We trained the models using the same dataset splits to have fair comparison between the results. However for the tree regression models we chose specific parameter grids, for DecisionTree and RandomForest Regressor we created a parameter grid to use with gridsearch to find the best parameters for the Max-depth, Minimal samples of splits and minimum samples of leafs for our tree. We did this so the best set of performing parameters would be selected during training before we fit our models. Also, implemented random state to 42 to consider consistent data and results.

Our classification models utilized the variables we used when we trained and split the data. When fitting each model we used the same parameters of data from the training split alongside unique parameters towards some classification models. For example, in our KNN model we set k to equal 5, for QDA we set the regular parameter to 0.5, and for LDA we used the default method for this algorithm. For our decision tree model, we set the random state to be equivalent to 42 in order to ensure a consistent seed of data for the model. For logistic regression, we encountered some issues during execution time which will further be discussed in the “Model Optimization” section. All of our models we’re able to execute however, we encountered issues such as a limit error for logistic regression due to the maximum iterations currently set by default as the best model wasn’t found for logistic regression under the default maximum iteration.

3.6 Model Evaluation

Once the model is trained, the next step is to evaluate its performance on a validation dataset.

The evaluation of our regression was performed on a test-set that was not previously used rather than a separate validation set. However a grid-search algorithm was utilized during training for the tree models to find the best splits for parameters. The utilization of the separate set was implemented to prevent overfitting and data-leaking from seen data. The metrics we used to evaluate our regression models was mean-squared-error (MSE), root-mean-squared-error (RMSE) and R^2 . We used these metrics as it served as a basis to test for our models average error, interpretability and to confirm the scope of capturing the data effectively. Metrics are further discussed in sec. 4.3.

To evaluate our classification models we analyzed the accuracy scores and used k-fold cross validation to further analyze a set of accuracy scores of each model in order to compare and find a model with the highest accuracy score/precision. After conducting k-fold cross validation where $k = 20$, we found that most of the accuracy scores were close to the accuracy scores calculated previously for

each model as displayed in figure #. We found that our decision tree model yielded the highest accuracy score and chose to optimize that model furthermore in order to reach an accuracy score of 80% or higher.

3.7 Model Optimization

Based on the results of the model evaluation, the next step is to optimize the model to improve its performance. This involves making changes to the model's architecture, tuning the hyperparameters, and retraining the model.

Throughout the process of improving our model in predicting delay in minutes we made several adjustments to retrain our model. Such as feature-engineering utilizing existing features to create new features. Implementing a grid-search for our regression tree models to find optimal hyperparameters. Both emerged in significant improvements in metrics and predictiveness.

For our decision tree classification model, in order to further optimize the model we utilized grid search cross validation for hypertuning in order to find the best decision tree model and parameters. By tuning the maximum depth range and minimum sample split parameters for grid search, we were able to find a model that would yield an accuracy score of 80% through trial and error. We've also tuned the number of cross validations in grid search and found that 20 folds yield an accuracy score of 80% or higher.

3.8 Final Model Building

Once the model is optimized, the final step is to fit the model and report final test error for each model

Present the information with support Screen Shots/ Figures (Each Figure must be numbered and Description of Figure must be provided)

Shown in section 4.0 onward and see list of figures:

Chapter 4

Results

To compare different machine learning models in the results section, here are some steps to consider:

4.1 Description of the Models

Start by providing a brief description of each model, including its purpose, algorithm type, and any relevant parameters or features used.

Regression Models:

Linear Regression A supervised machine learning algorithm that is used to predict the output as a continuous value based off quantitative predictors. This is done to model the relationship between the independent and dependent variable. The model uses a linear line to create the relationship and the purpose in our projects context was to serve as a baseline for comparisons to other models.

Decision Tree Regressor A supervised machine learning algorithm that uses a tree structure to split the data into separate nodes to predict a continuous value. The purpose of this structure is to split decisions into smaller nodes until it reaches a leaf-node to output a final continuous value. In our projects context the specific features were max-depth, minimal samples of splits and minimum samples of leaf.

Random Forest Regressor A supervised machine learning algorithm that is similar to Decision Tree's uses a tree structure but combines multiple tree's to output the response as a continuous value. This is useful for capturing non-linear data by spreading subsets of the data to train on many trees. By doing this it aided in predicting the minute delay as it prioritized stability to get results. The specific parameters we used here were max-depth, minimal samples of split, minimum samples of leaf and max_features.

For classification, we used several models as we wanted to compare the accuracy scores of this model and select the model with the highest accuracy score to optimize and fine tune. The classification models we used are displayed in the table below.

Classification Models:

Linear Discriminant Analysis (LDA) A supervised machine learning algorithm that sets up linear decision boundaries.

Quadratic Discriminant Analysis (QDA) A supervised machine learning algorithm that sets up non-linear decision boundaries.

Logistic Regression is A machine learning algorithm that predicts the probability of categorical outcomes based on input data. **Decision Tree (Classification Tree)** A machine learning method that categorizes data and converges upon certain points.

K-Nearest Neighbour (KNN) An algorithm that checks a data point's neighbors to make a decision. Classifying a point of data based on the nearest k points or data, this eventually results in a boundary or several boundaries.

4.2 Performance Metrics

Describe the performance metrics used to evaluate the models, such as RMSE, MSE, accuracy, precision, recall, F1 score, or area under the receiver operating characteristic curve (AUC-ROC), etc based on the model class.

The performance metrics we used to evaluate our models was RMSE, MSE and R^2 .

MSE was used to compare our models predicted results versus the actual results to see our models average error. The RMSE was utilized to interpret the error in minutes, so clear conclusions of our average models deviation from the actual data. As RMSE provides a interpretable way to measure the deviations for predictions of delay in minutes, allowing us to perceive how our data differs. R^2 measured how our model explains the data and how well our model explains the variability within the data.

Furthermore, to evaluate our classification models we went ahead and compared performance metrics such as the accuracy between each model. Initially, these values were very low ranging from 0% to 20% accuracy due to the immense outliers present within our dataset. By restricting our outliers as mentioned previously in section 2.2 "Data Processing", we managed to yield the accuracy results displayed in table #. We found that our decision tree model had the highest accuracy, however we wanted to find a model yielding an accuracy of 80%. Therefore, we'd need to further optimize these models and tune parameters in order to increase accuracy.

4.3 Results Table

Create a results table that presents the performance metrics for each model in a clear and concise format. The table should include the name of the model, the performance metric, and the value of the metric. It can also be helpful to include the standard deviation or confidence intervals for each metric.

Classification Models	Accuracy (%)
Linear Discriminant Analysis (LDA)	0.68
Quadratic Discriminant Analysis (QDA)	0.67
Logistic Regression	0.68
Decision Tree	0.78
K-Nearest Neighbour (KNN)	0.70

Figure 4.1: Displays accuracy and precision scores for each classification model.

Regression models	Mean Square Error	Root Mean Square Error	R ²
Linear Regression	83.66	9.15	0.03
Decision Tree Regression	61.16	7.82	0.29
Random Forest Regression	50.68	7.12	0.41

Figure 4.2: Displays MSE, RMSE, and R² for each regression model.

4.4 Interpretation of the Results

Provide an interpretation of the results, highlighting any significant differences between the models. Discuss which model performed the best and why, as well as any limitations or potential sources of error.

Amongst our regression, the random Forest regressor model performed the best in every metric with a lower mse, rmse and a higher r^2 . The reason is due to the averaging out of tree's to come to a prediction. Additionally, this model robustness against noise is another strong factor given the high variance in the dataset. Significant difference can be observed between the linear regression model and the tree-based regression models highlighting the data is not linearly separable and is too complex to be captured using a linear relationship. The observed differences from decision tree and random forest could potentially be from random-forest averaging predictions across many trees, reducing variances. This factor is an explanation behind the higher r^2 as the dataset contained high variability.

Amongst our classification models, the decision tree model performed the best as it yielded the highest accuracy precision score amongst all the other models utilizing the training and testing.

After conducting 20-fold cross validation, this was even more apparent as most of the accuracy scores for the decision tree model were higher than the accuracy scores present in the sets present within the other models.

4.5 Visualization

Provide visualizations to support the interpretation of the results, such as bar charts or box plots. These can help to illustrate the differences between the models and make the results more accessible to readers.

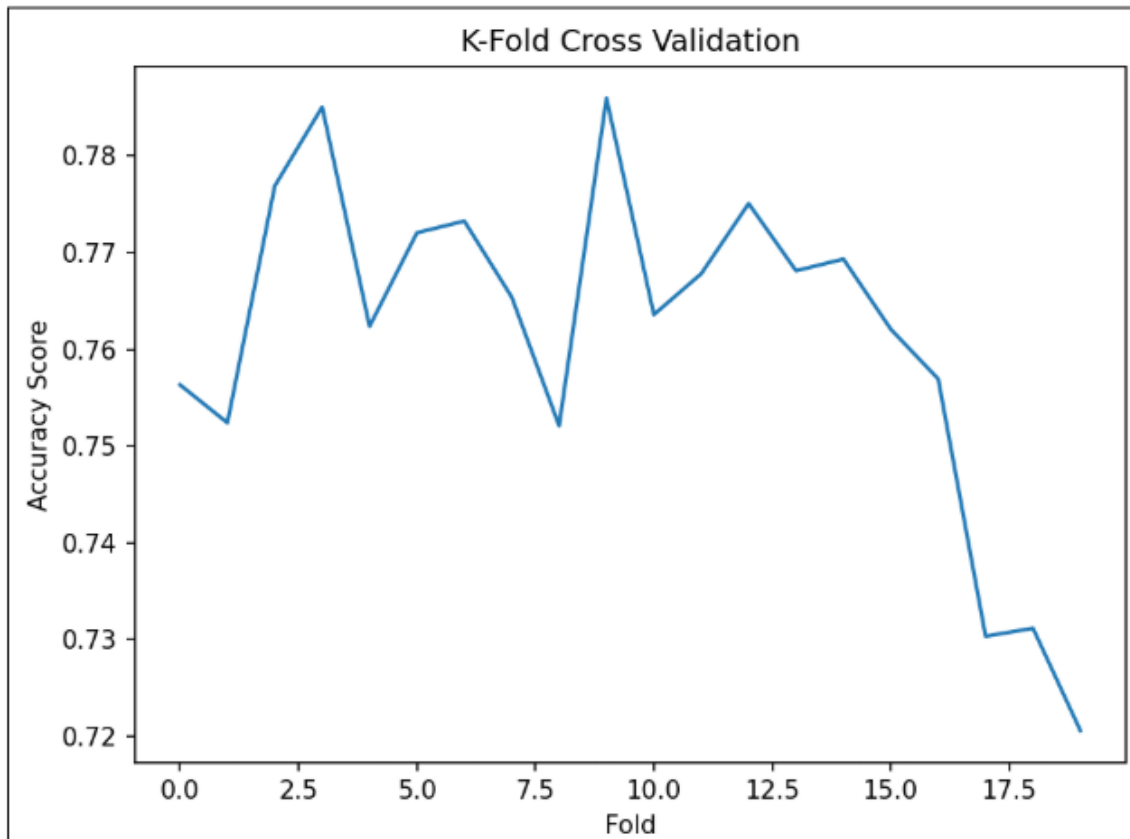


Figure 4.3: 20-fold cross-validation results for the tuned decision tree model using grid search.

Months represent the months of January to December (1 = January, 12 = December). Each month is classified as true or false, representing the month to have rush hours 50% or more of the time. False representing that rush hour occurs less than 50% of the time. Rush hour is correlated with delay risk.

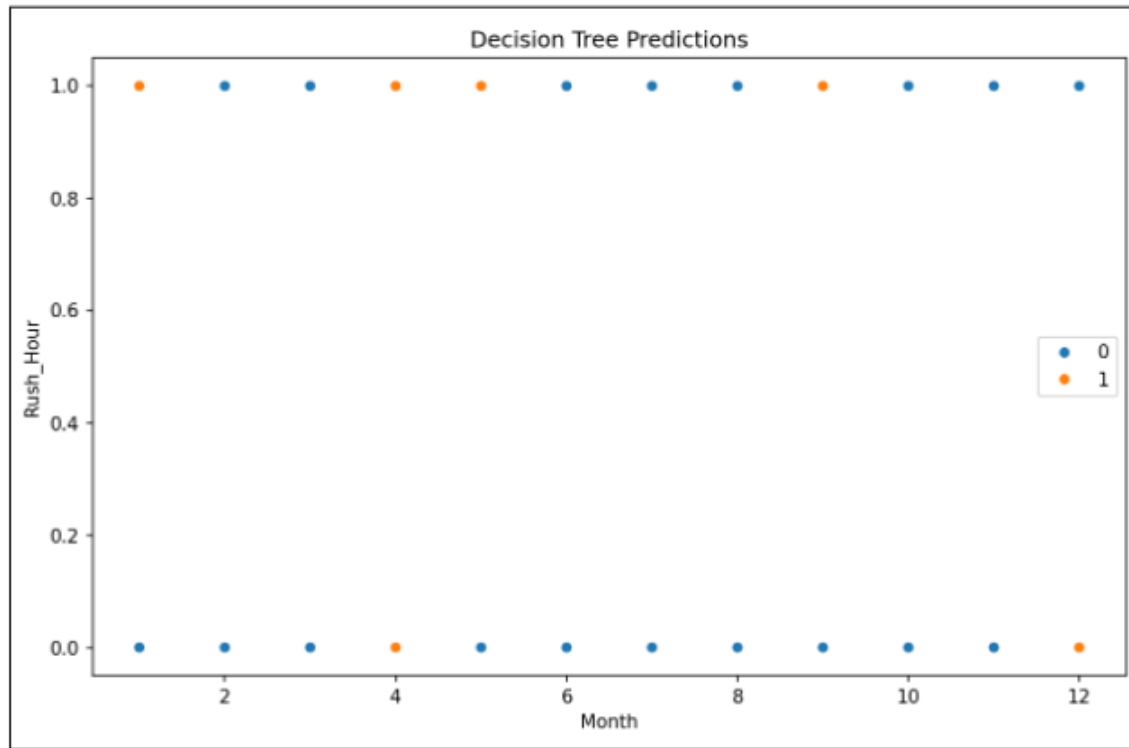


Figure 4.4: Decision Tree Predictions

Based on our results for our tuned decision tree model, we can see that the decision tree model yields an accuracy of approximately 80.59%. The optimal number of folds appears to range from 8 to 10. Overall, this model yields the optimal target accuracy score compared to the other classification models.

Regression models:

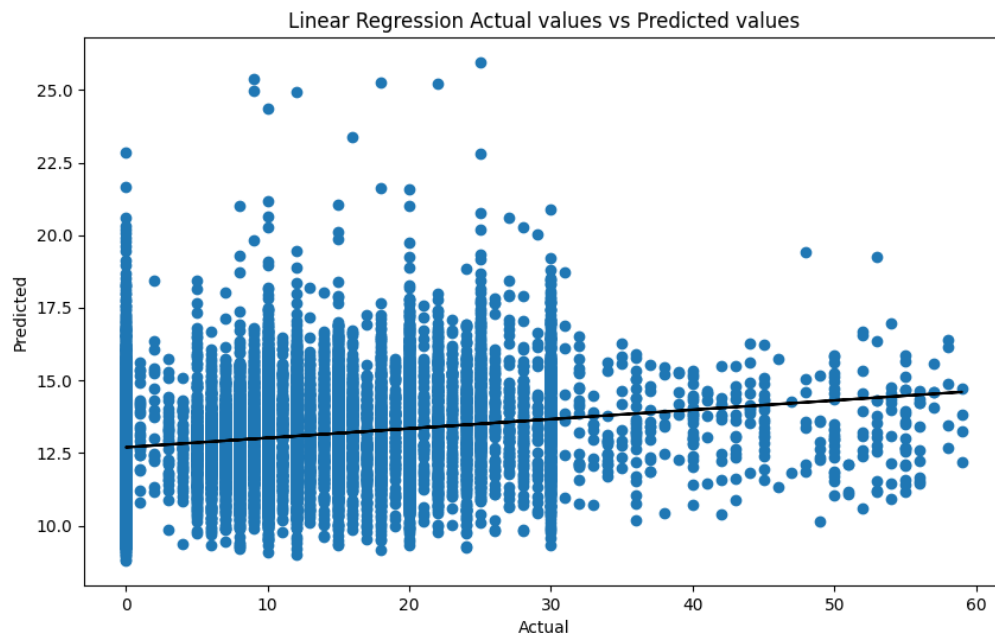


Figure 4.5: Linear Regression Actual vs Predicted Scatterplot

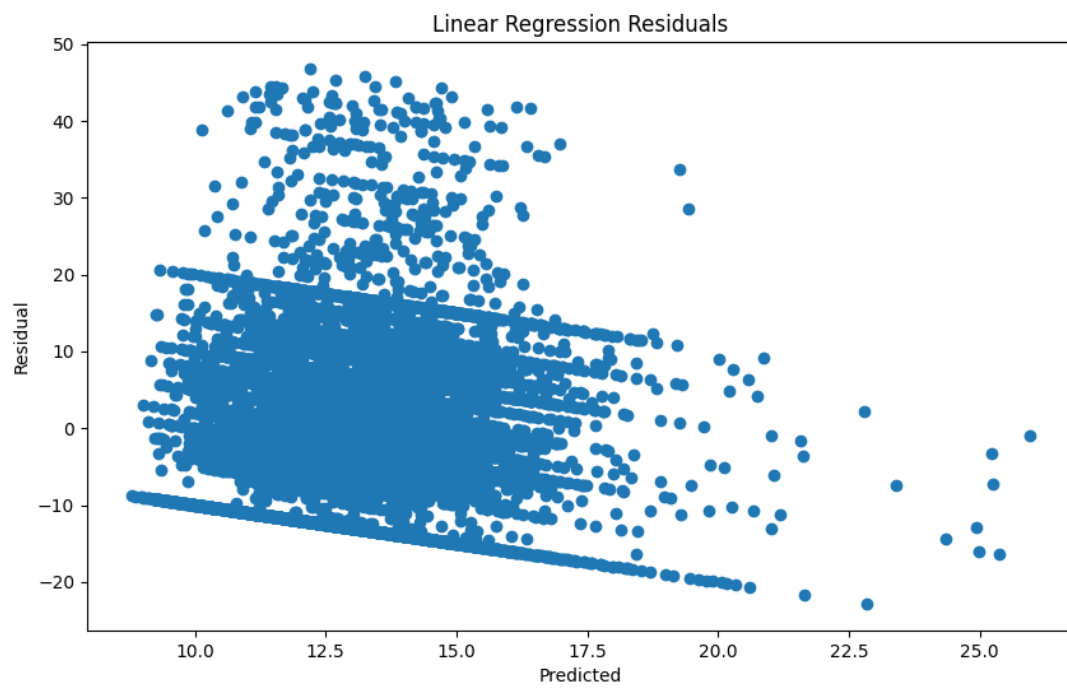


Figure 4.6: Linear Regression Residuals

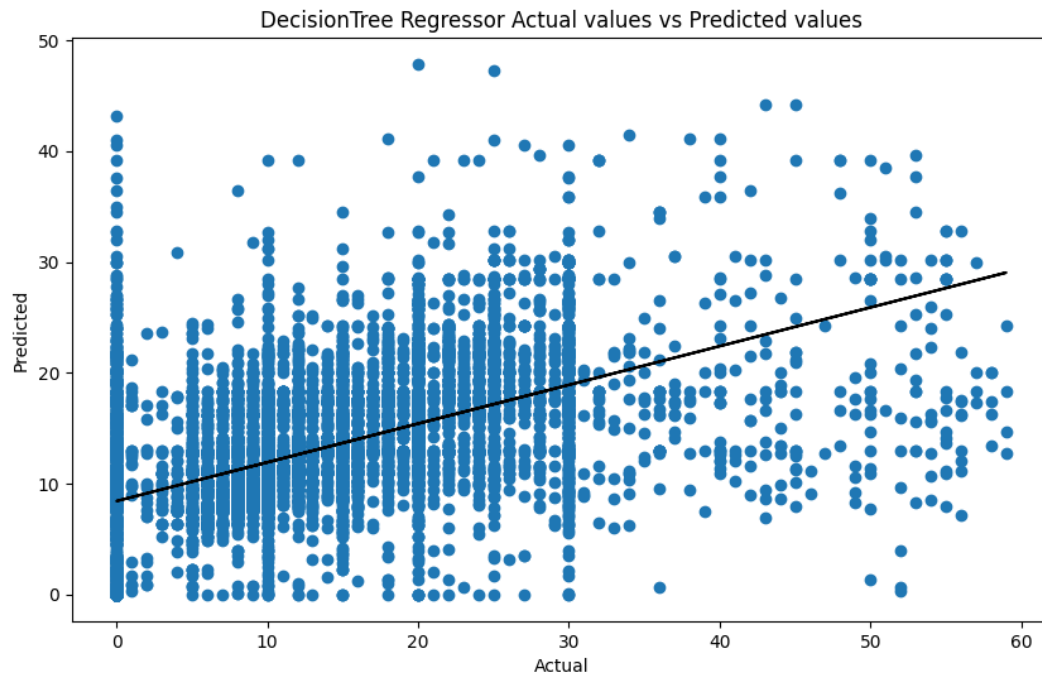


Figure 4.7: Reg-Decision Tree Actual vs Predicted Scatterplot



Figure 4.8: Reg-Decision Tree Residuals

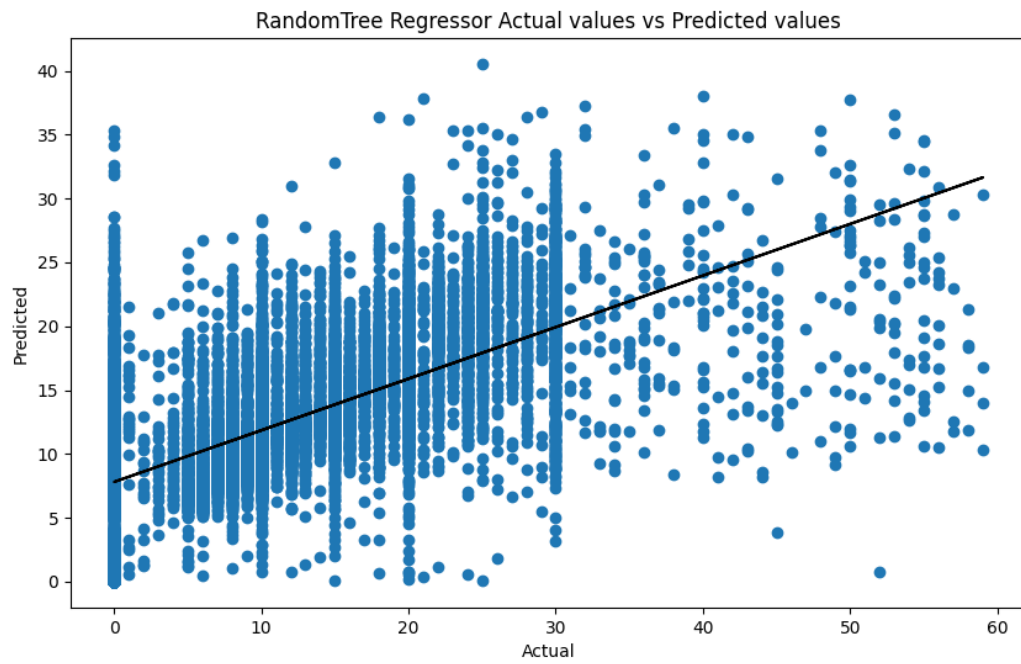


Figure 4.9: Random-reg Actual vs Predicted Scatterplot

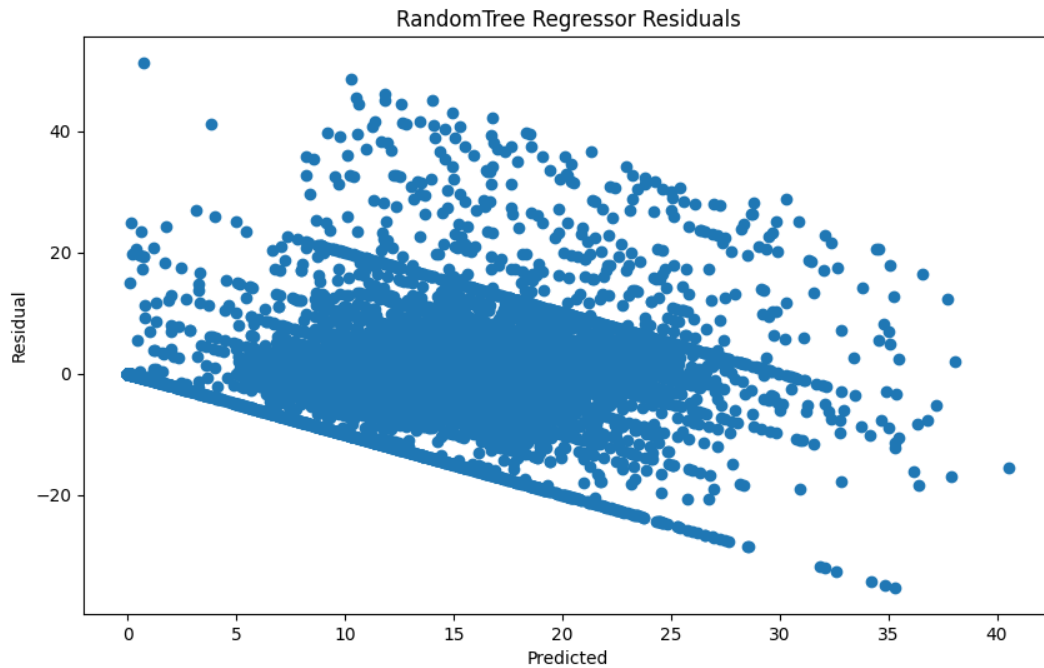


Figure 4.10: Random-reg Tree Residuals

4.6 Sensitivity Analysis

Conduct a sensitivity analysis to test the robustness of the models. This can involve varying the parameters or features used in the models and re-evaluating their performance.

When testing our models when we only keep the predictors at $\text{delay_minutes} < 30$ min. For the regression models we see that the linear regression model still performs the worst compared to Figure 4.2 however the error does improve alongside the other models. Highlighting the model benefiting from less noise. Consistently like the previous figure the other models improves in a steady rate, random forest remaining top performer. Due to the ensembling techniques like bagging reducing the variance. Additionally, benefits because of a less exhaustive dataset since it contains less features.

```

LinearRegression results:
Mean Square Error: 49.636541174735505
Root Mean Square Error: 7.045320516111066
R-Square: 0.03321427337693161

DescisionTree results:
Improved score: 0.357575695346605
Improved parameters: {'max_depth': 15, 'min_samples_leaf': 4, 'min_samples_split': 16}
Mean Square Error: 33.9741494201618
Root Mean Square Error: 5.828734804411829
R-Square: 0.33827535206482184

RandomTree results:
Improved score: 0.46137763429950895
Improved parameters: {'max_depth': None, 'max_features': None, 'min_samples_leaf': 3, 'min_samples_split': 10}
Mean Square Error: 27.68423855286793
Root Mean Square Error: 5.261581373776132
R-Square: 0.4607858232683675

```

Figure 4.11: Reg-Sens

Furthermore, when testing our decision tree model we initially had to tune the hyperparameters in order to gradually increase the accuracy score which was approximately 78%. To increase this score, we went ahead and changed the number of folds from 10 to 20 and increased the maximum depth range and minimum sample split lists within our grid search cross validation. The following figures displays the results of the decision tree model with different maximum depth range and minimum sample split list parameters.

	Parameters used in Figure 4.13	Parameters used in Figure 4.14
<i>maximum_depth_range</i>	[None, 5, 10, 15, 20]	[None, 5, 10, 15, 20, 35, 40, 45, 50]
<i>minimum_sample_list</i>	[2, 4, 8, 16]	[2, 4, 8, 16, 32, 64]

Figure 4.12: Displays the parameters used for *maximum_depth_range* and *minimum_sample_list* in each figure undergoing 20-fold cross validation grid search

```

Grid Search for Decision Tree Model

Improved score: 0.791302489790262
Improved parameters: {'max_depth': 15, 'min_samples_split': 16}
Best Decision Tree Model: DecisionTreeClassifier(max_depth=15, min_samples_split=16, random_state=42)
Best Decision Tree Parameters: {'max_depth': 15, 'min_samples_split': 16}

```

Figure 4.13: First set of parameters, results of grid search using a specific set of parameters mentioned in Table

```
Grid Search for Decision Tree Model  
  
Improved score: 0.8055561167199107  
Improved parameters: {'max_depth': 20, 'min_samples_split': 64}  
Best Decision Tree Model: DecisionTreeClassifier(max_depth=20, min_samples_split=64, random_state=42)  
Best Decision Tree Parameters: {'max_depth': 20, 'min_samples_split': 64}
```

Figure 4.14: Second set of parameters, results of grid search using a specific set of parameters mentioned in Table

Chapter 5

Conclusion

Conclude by summarizing the key findings of the analysis, discussing the implications of the results, and suggesting areas for future research.

Key findings of the analysis is that the dataset itself contains many large outliers however our models have improved performance when we create specific conditions to reduce the total number of feature. In terms of the regression models, clear from analyzing the linear to the tree models the data is non-linear which benefits from non-linear modeling approaches. The random forest model was the best performer because of how it handles variance through averaging, handling noise and variability within the dataset. Additionally, predictability improved significantly from feature engineering, emphasizing the importance of good features and creating new features. Overall, random forest saw the most improvements but for future research into this problem I believe that it can improve significantly from more transformations within the data.

Furthermore, the classification model we found to be the best and most accurate was the decision tree model with an accuracy score of approximately 80% that was trained under 20-fold grid search. This model classifies each month with a delay risk, 1 being the delay risk being 50% or greater and 0 being otherwise. In other words, 1 represents months with a common occurrence of delay while 0 represents months with a less common delay.

References

- Knuth, D. E. (1984). Literate programming. *Comput. J.*, 27(2), 97–111. <https://doi.org/10.1093/comjnl/27.2.97>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Sehra, S. S. (2026). *Machine-learning google colab notebooks*. <https://colab.research.google.com/>.